MOCKING

Replace real dependencies with **controlled test implementations.**



mock_test.go

```go
type UserRepository interface {
    Find(id int) (User, error)
}

type MockRepository struct{}
```

- What mocking is
- Why interfaces
- Manual mocks
- When to mock

1/10 — THE PROBLEM

@go_evangelist

Components Rarely Work in Isolation

- A database
- An HTTP API
- A cache
- A message queue
- A file system
- A payment provider
- An authentication service

TESTING AGAINST REAL EXTERNAL SYSTEMS:

- Tests become **slower**
- Failures depend on **network**
- Databases **must be prepared**
- External services **may change**
- Tests become **non-deterministic**

REAL DEPENDENCY CHAIN

HTTP Request → Authentication → Load Configuration → Connect Database → Run SQL Query → Return Result

A mock reduces it to: Mock → Return User

ADVANTAGES OF MOCKING

- Fast
- Deterministic
- Isolated
- Repeatable
- Flexible



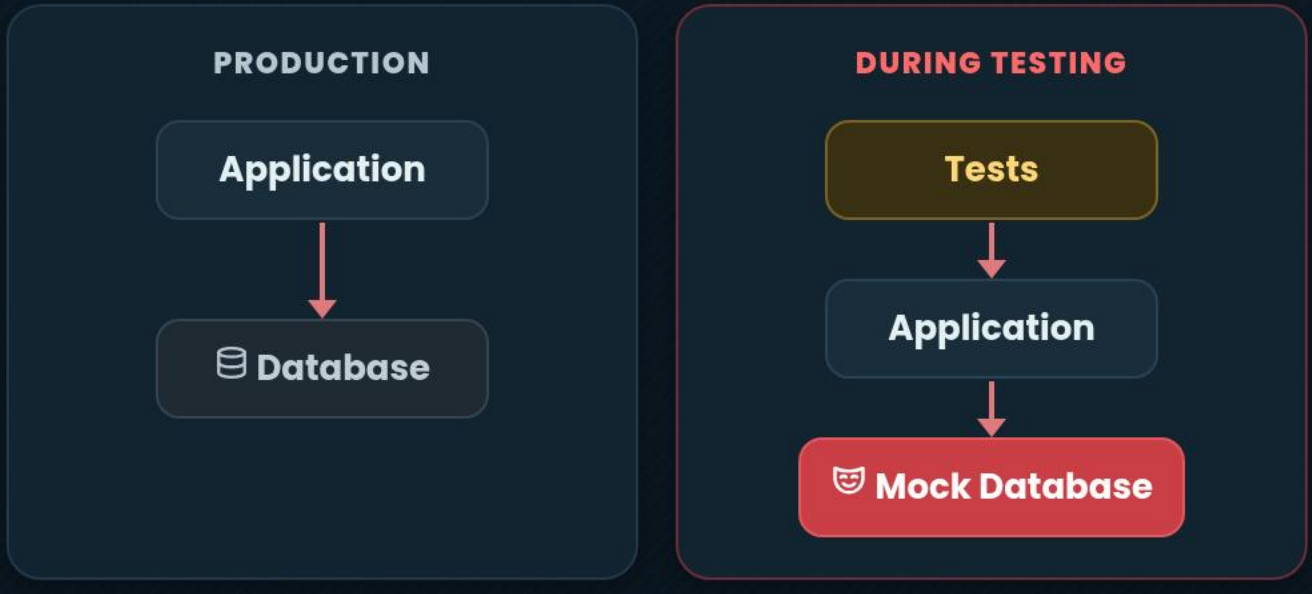
Go Testing Handbook · Chapter 8: Mocking

GO

@go_evangelist

What Is Mocking?

A **Mock** is an object that replaces a real dependency during testing. Instead of the real dependency, the code interacts with a **controlled test implementation**.





The application does not know the difference.

It simply communicates through an **interface**.

The dependency is replaced with predictable behavior.

3/10 — INTERFACES

@go_evangelist

The Role of Interfaces

In Go, mocking is almost always based on **interfaces**. The service depends on the interface, not a concrete implementation.

service.go

```go
type UserRepository interface {
    Find(id int) (User, error)
}

type UserService struct {
    repo UserRepository
}
```



DURING PRODUCTION

Application → UserService → SQLRepository → Database

DURING TESTING

Test → UserService → MockRepository → Static Result

The service code itself **does not change.**

Go Testing Handbook · Chapter 8: Mocking

GO

4/10 — DEPENDENCY INJECTION

@go_evangelist

Inject Your Dependencies

Mocks become possible because dependencies are **injected** rather than created internally.

Good design

```
type UserService struct {
    repo UserRepository
}
```

Allows **different implementations**.

Poor design

```
type UserService struct {
    db *sql.DB
}
```

Tightly couples the service to a specific database.



Dependency Injection + interfaces are fundamental for effective mocking in Go.

Go Testing Handbook · Chapter 8: Mocking

GO

5/10 — MANUAL MOCKS

@go_evangelist

Manual Mocks in Go

One of Go's strengths: many mocks can be **written manually**.

clock_test.go

```go
type MockClock struct{}

func (m *MockClock) Now() time.Time {
    return time.Date(2026, 1, 1, 0, 0, 0, 0, time.UTC)
}
```



Always returns the same time: **ideal for testing**.

email_test.go

```go
type EmailSender interface {
    Send(to, subject, body string) error
}

type MockEmailSender struct{}

func (m *MockEmailSender) Send(to, subject, body string) error {
    return nil
}
```

Never contacts an external system. **Predictable and deterministic.**

Often no external library is required.

Go Testing Handbook · Chapter 8: Mocking

GO

6/10 — VERIFY & SIMULATE

@go_evangelist

Verifying Behavior & Different Results

Was **Send()** called? Check **how** the dependency was used:

email_test.go

```go
type MockEmailSender struct {
    Called bool
}

func (m *MockEmailSender) Send(to, subject, body string) error {
    m.Called = true
    return nil
}
```

```go
if !mock.Called {
    t.Fatal("expected Send to be called")
}
```

RETURN DIFFERENT RESULTS:

Success	`return nil`
Failure	`return errors.New("network error")`
Timeout	`return context.DeadlineExceeded`
Auth error	`return ErrUnauthorized`

Go Testing Handbook · **Chapter 8: Mocking**

GO

7/10 — CONFIGURABLE MOCKS

@go_evangelist

Configurable Mocks

mock_test.go

```go
type MockRepository struct {
    User User
    Err  error
}

func (m *MockRepository) Find(id int) (User, error) {
    return m.User, m.Err
}
```

Success: return a user

```go
mock := MockRepository{
    User: User{ID: 1, Name: "Alice"},
}
```

Or: return an error

```go
mock := MockRepository{
    Err: errors.New("database unavailable"),
}
```

The mock stays **reusable**, and each test describes its own scenario clearly.

Go Testing Handbook · Chapter 8: Mocking

GO

8/10 — TEST DOUBLES

@go_evangelist

Types of Test Doubles

Dummy	Passed only to satisfy an API.
Stub	Returns predefined values.
Fake	A simplified working implementation (for example, an in-memory repository).
Mock	Verifies interactions and expectations.
Spy	Records calls for later inspection.

Go developers frequently implement these concepts using ordinary structs.

Avoid Over-Mocking

Service → Mock → Mock → Mock → Mock → Mock

Eventually, the unit test verifies only that the mocks behave as configured. **Very little real application logic is exercised.**

→ **Brittle tests that are difficult to maintain.**



Go Testing Handbook · Chapter 8: Mocking

GO

@go_evangelist

To Mock or Not?

Mock external deps

- HTTP clients
- Databases
- Message queues
- Payment providers
- SMTP servers
- File storage services
- Cloud APIs
- Authentication providers
- Time providers
- Random number generators

Usually NOT

- Simple value objects
- Standard library types
- Business entities
- Pure functions
- Immutable data structures
- Small utility functions



Interfaces Should Be Small

× Hard to mock

```
type Repository interface {
    Create(...)
    Update(...)
    Delete(...)
    Find(...)
    FindAll(...)
    Search(...)
    Count(...)
    Close(...)
}
```

✓ Small and focused

```
type UserFinder interface {
    Find(id int) (User, error)
}
```

A manual mock must implement **every method**, even if a particular test only needs one.

@go_evangelist

Key Takeaways

- Mocking isolates the component under test.
- Interfaces are the foundation of mocking in Go.
- Dependency Injection makes mocking straightforward.
- Manual mocks are often sufficient and idiomatic.
- Mocks simulate both success and failure.
- Small interfaces are easier to mock and maintain.
- Mock external dependencies, not business logic.
- Complement mocks with integration tests.

COMMON BEGINNER MISTAKES

- × Mocking everything
- × Large interfaces just for testing
- × Ignoring integration tests

BEST PRACTICES

- Verify meaningful interactions
- Keep mocks deterministic
- Avoid unnecessary complexity



NEXT · CHAPTER 9

Best Practices →

Recommended workflow, useful `go test` commands & resources.